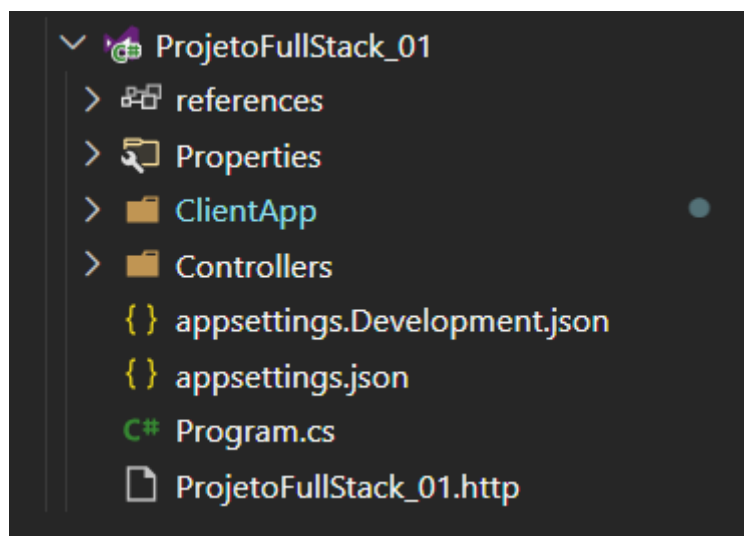


Angular + .Net

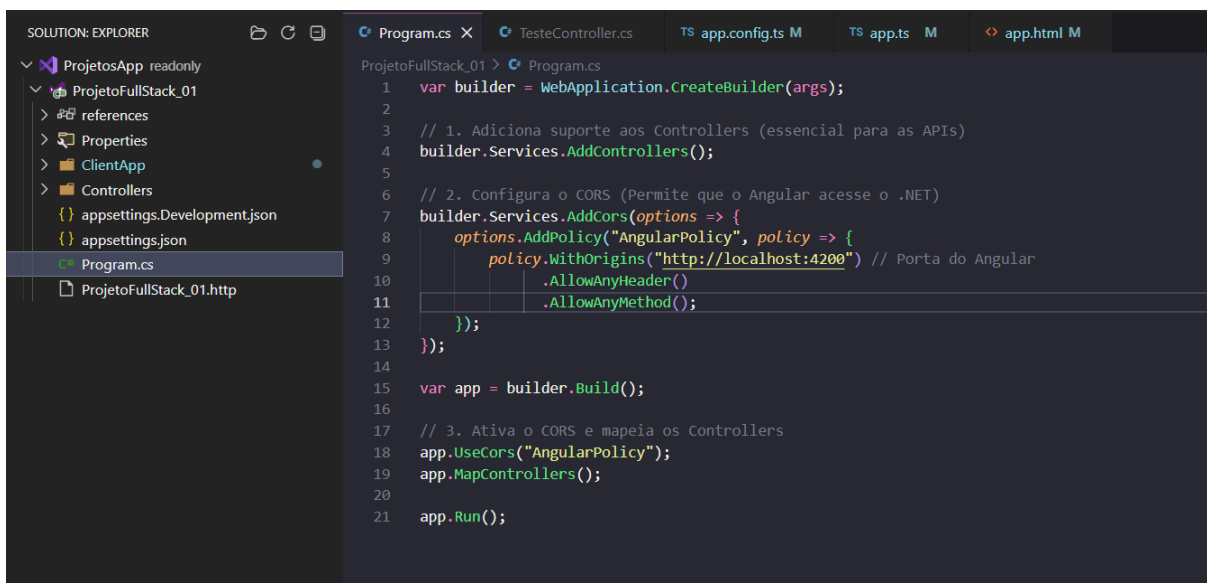
Primeiros Passos

Primeiro criaremos o nosso Projeto API .Net nesse caso o chamamos de **ProjetoFullStack_01** e dentro dele criaremos um Projeto Angular chamado **ClientApp**



📁 Caminho do Arquivo : **ProjetoFullStack_01** / [Program.cs](#)

→ Aqui configuramos e fazemos a integração com .Net + Angular



1. `builder.Services.AddControllers();` (Linha 4)

- **O que faz:** Habilita o suporte para que o .NET reconheça as classes dentro da pasta `Controllers`.
- **Por que é importante:** Sem essa linha, o .NET não saberia o que fazer quando chegasse uma requisição para o seu `TesteController`. É aqui que ele ativa o motor das APIs.

2. `builder.Services.AddCors(...)` (Linhas 7 a 13)

Esse é o ponto mais crítico da integração. **CORS** significa *Cross-Origin Resource Sharing* (Compartilhamento de Recursos entre Origens).

- `AddPolicy("AngularPolicy", ...)`: Você está criando uma "regra de segurança" e dando o nome de "AngularPolicy" para ela.
- `WithOrigins("http://localhost:4200")`: Aqui você está dando a "chave da casa" especificamente para o seu Angular. Você está dizendo ao .NET: *"Se alguém do localhost:4200 me chamar, eu confio nele"*.
- `AllowAnyHeader()` e `AllowAnyMethod()`: Você está permitindo que o Angular envie qualquer tipo de informação (Headers) e use qualquer método (GET, POST, DELETE, etc.).

3. `var app = builder.Build();` (Linha 15)

- **O que faz:** Encerra a fase de configuração (onde você estava adicionando ingredientes) e "constrói" o servidor de fato. A partir daqui, você começa a configurar como os pedidos serão tratados.

4. `app.UseCors("AngularPolicy");` (Linha 18)

- **O que faz:** Ativa a regra que você criou lá em cima.
- **Atenção:** A ordem aqui importa! Você deve ativar o CORS **antes** de mapear os controllers para que a permissão seja verificada antes de qualquer outra coisa. Se você esquecer essa linha, o navegador vai dar aquele erro vermelho bloqueando o Angular.

5. `app.MapControllers();` (Linha 19)

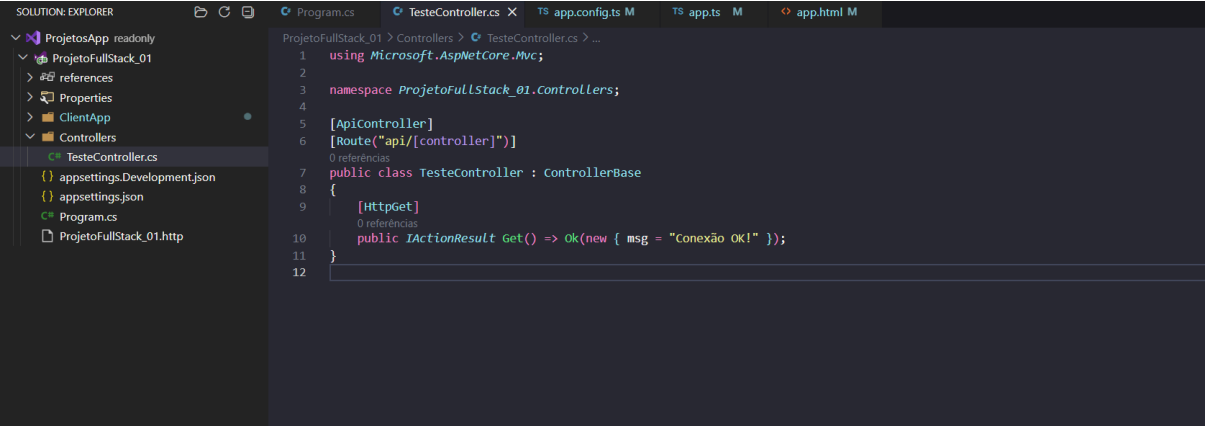
- **O que faz:** É o comando final que "liga os pontos". Ele vasculha seu projeto atrás de rotas como `[Route("api/[controller]")]`.
- **Na prática:** Ele cria o mapa de estradas. Quando o Angular pede `/api/teste`, é essa função que sabe exatamente em qual arquivo e em qual método o código deve entrar.

6. `app.Run()` ; (Linha 21)

- **O que faz:** Dá o sinal de partida. O servidor trava o terminal e fica "ouvindo" as portas (como a 5231) esperando alguém bater na porta.

📁 Caminho do Arquivo : `ProjetoFullStack_01 / Controllers / TesteController.cs`

→ Aqui é onde criaremos a nossa API



```
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace ProjetoFullstack_01.Controllers;
4
5 [ApiController]
6 [Route("api/[controller]")]
7 public class TesteController : ControllerBase
8 {
9     [HttpGet]
10    public IActionResult Get() => Ok(new { msg = "Conexão OK!" });
11 }
12
```

1. A Identidade da API (Linhas 5 e 6)

- **[ApiController]:** Esse atributo avisa ao .NET que esta classe não é uma página web comum, mas sim uma **API**. Ela vai responder dados (como JSON) em vez de arquivos HTML.
- **[Route("api/[controller]")]:** Aqui você define o endereço da API. O termo `[controller]` é um atalho que assume o nome da classe sem a palavra "Controller".
 - **Resultado:** O endereço para chamar esse código será `http://localhost:5231/api/teste`.

2. A Herança (Linha 7)

- **: ControllerBase:** Sua classe "herda" ferramentas prontas da Microsoft para lidar com requisições HTTP, como o comando `Ok()`, que usamos abaixo.

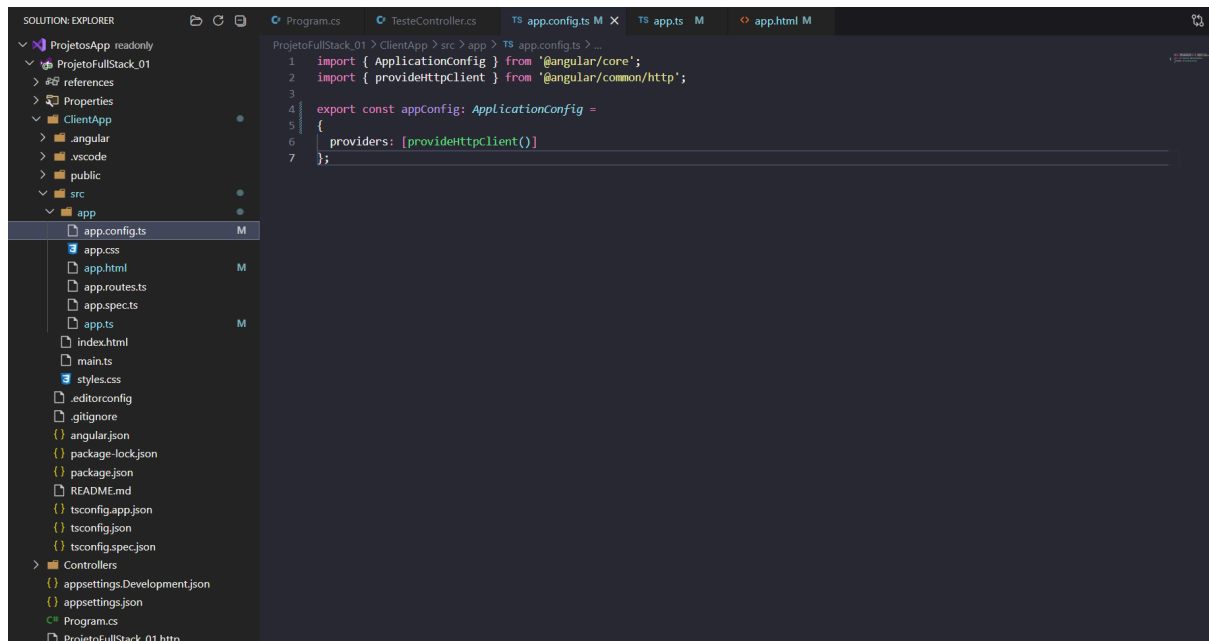
3. A Requisição GET (Linhas 9 e 10)

Aqui está a funcionalidade que você criou:

- **[HttpGet]**: Define que este método só responde a requisições do tipo **GET** (usado para buscar/leitura de dados). É o método que o navegador ou o Angular usam por padrão para pedir informações.
- **public IActionResult Get() => Ok(...)**:
 - O **Ok()** gera um código de sucesso **HTTP 200**.
 - O conteúdo dentro dele **new { msg = "Conexão OK!" }** cria um **objeto anônimo** que o .NET transforma automaticamente em **JSON**.

📁 **Caminho do Arquivo** : ProjetoFullStack_01 / ClientApp / src / app / app.config.ts

→ Configuração Http Angular



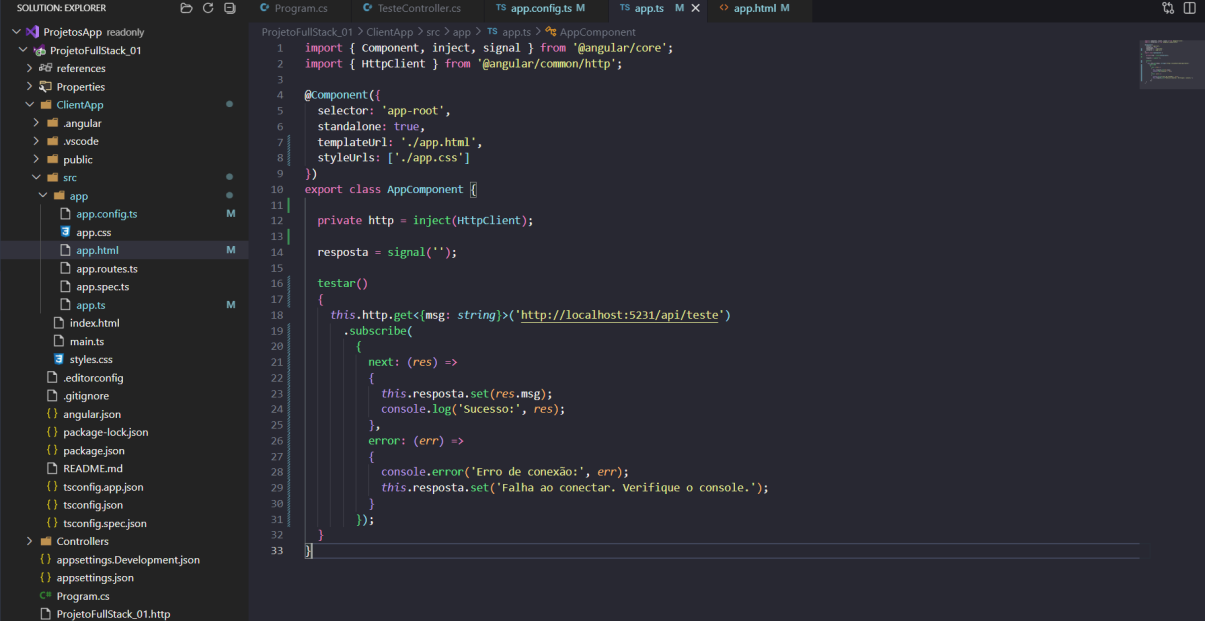
1. **provideHttpClient()** (Linha 6)

Este é o ponto principal da sua integração.

- **A função**: Ele registra o serviço de cliente HTTP no sistema de **Injeção de Dependência** do Angular.
- **O que acontece na prática**: Quando você coloca **private http = inject(HttpClient)** lá no seu arquivo **app.ts**, o Angular só consegue entregar esse "motor" de requisições porque você o declarou aqui primeiro no **app.config.ts**.

📁 Caminho do Arquivo : ProjetoFullStack_01 / ClientApp / src / app / app.ts

→ Neste arquivo é onde nós fazemos a nossa requisição Http “API que nós criamos”, essa requisição nos entregará Dados em **JSON** então nós vamos traduzir para linguagem **Typescript** cujo essas informações serão preparadas e enviadas para o **Front-End** ou melhor para a página de renderização **app.html**



```
1 import { Component, inject, signal } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3
4 @Component({
5   selector: 'app-root',
6   standalone: true,
7   templateUrl: './app.html',
8   styleUrls: ['./app.css']
9 })
10 export class AppComponent {
11
12   private http = inject(HttpClient);
13
14   resposta = signal('');
15
16   testar()
17   {
18     this.http.get<{msg: string}>('http://localhost:5231/api/teste')
19       .subscribe(
20         {
21           next: (res) =>
22             {
23               this.resposta.set(res.msg);
24               console.log("Sucesso:", res);
25             },
26           error: (err) =>
27             {
28               console.error("Erro de conexão:", err);
29               this.resposta.set('Falha ao conectar. Verifique o console.');
```

1. A Injeção do "Telefone" (Linha 14)

- **private http = inject(HttpClient);**: Aqui você está pegando aquela ferramenta que habilitamos no **app.config.ts**. É como se você estivesse instalando a linha telefônica dentro deste componente para que ele possa fazer "ligações" externas.

2. O Pedido (A Requisição) (Linha 18)

- **this.http.get(...)**: Este é o momento exato em que o Angular envia um sinal pela rede até o seu servidor .NET. Ele está batendo na porta **http://localhost:5231/api/teste** e perguntando: *"Você tem alguma informação para mim?"*.

3. A Tradução: JSON para TypeScript (Linha 18)

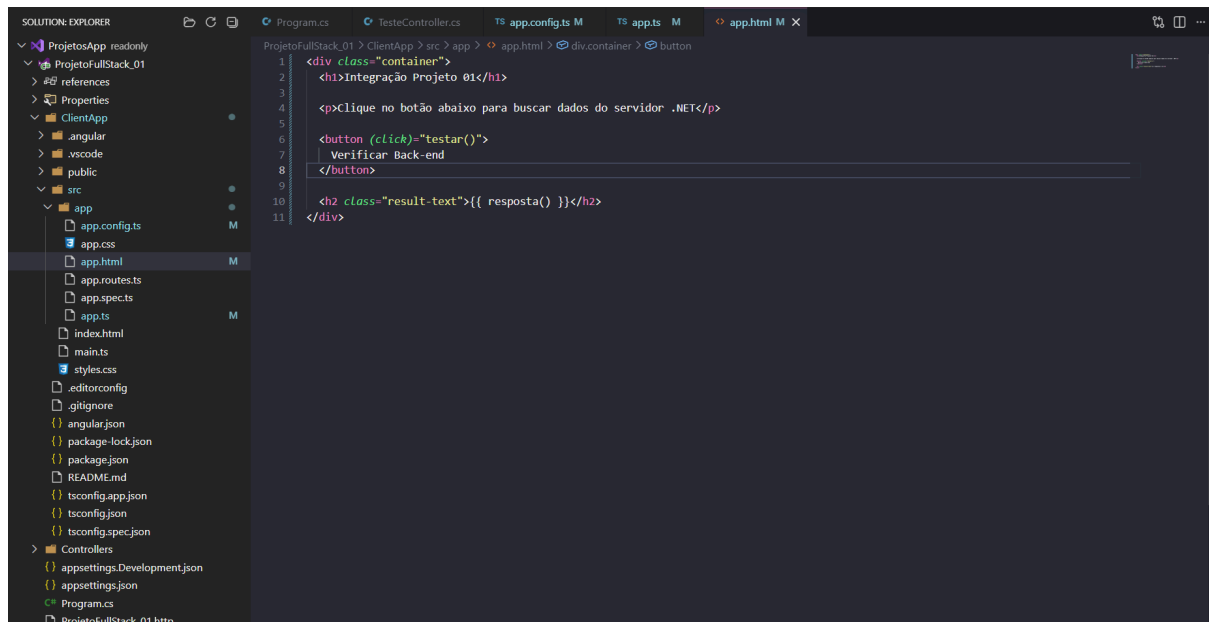
- **<{msg: string}>**: Isso é fundamental na Engenharia de Software. O Back-end envia um texto bruto (JSON). Com esse comando, você está **traduzindo** esse texto para a linguagem que o TypeScript entende. Você está dizendo: *"Eu espero receber um objeto que contenha uma propriedade chamada 'msg' do tipo texto"*. Isso evita erros de digitação e ajuda o VS Code a te dar sugestões.

4. O Recebimento e a Entrega (Linha 19)

- `.subscribe(res => this.resposta.set(res.msg)):`
 - O **subscribe** é como ficar esperando o retorno da ligação.
 - Assim que o .NET responde, o Angular pega o valor traduzido (`res.msg`) e o coloca dentro do **signal** chamado `resposta`.
 - Como o `resposta` é um Signal, ele avisa automaticamente o arquivo `app.html` para atualizar a tela sem que você precise recarregar a página.

📁 **Caminho dos Arquivos :** `ProjetoFullStack_01 / ClientApp / src / app / app.html`

→ Onde ocorre a renderização das informações que recebemos do arquivo `app.ts` v

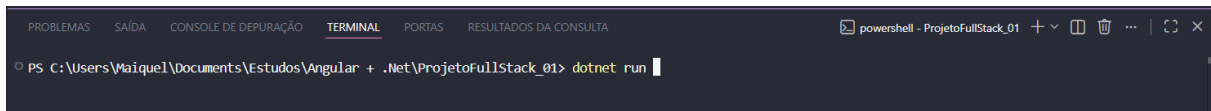


The screenshot shows the Visual Studio Code interface. On the left, the 'SOLUTION EXPLORER' pane displays the project structure for 'ProjetoFullStack_01'. The 'src' folder is expanded, showing the 'app' subfolder. The 'app.html' file is selected and its content is displayed in the main editor. The code in 'app.html' is as follows:

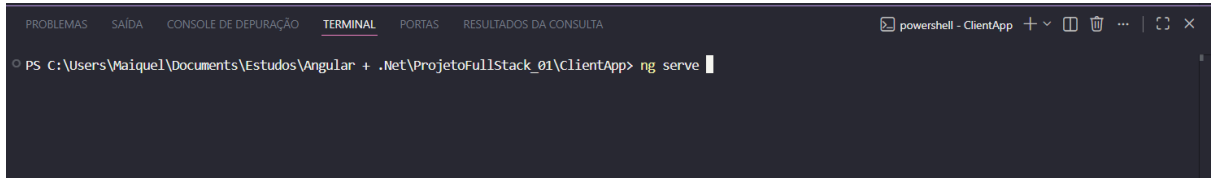
```
1 <div class="container">
2   <h1>Integração Projeto 01</h1>
3
4   <p>Clique no botão abaixo para buscar dados do servidor .NET</p>
5
6   <button (click)="testar()">
7     Verificar Back-end
8   </button>
9
10  <h2 class="result-text">{{ resposta() }}</h2>
11 </div>
```

O arquivo HTML é o responsável por **renderizar** (desenhar) as informações para o usuário final.

1. **O Gatilho:** O botão com `(click)="testar()"` é o que inicia todo o processo. Sem a interação do usuário, a API ficaria parada.
2. **A Exibição Dinâmica:** A tag `<h2>{{ resposta() }}</h2>` usa o que chamamos de **Interpolação**.
 - **Como funciona:** O Angular fica "vigiando" o Signal `resposta()`.
 - No momento em que o `app.ts` recebe o dado do .NET e atualiza o Signal, o HTML detecta essa mudança e **renderiza** o novo texto na tela instantaneamente, sem precisar recarregar a página.



```
PROBLEMAS SAÍDA CONSOLE DE DEPURAÇÃO TERMINAL PORTAS RESULTADOS DA CONSULTA powershell - ProjetoFullStack_01 + - [ ] ... | [ ] [X]
PS C:\Users\Maique1\Documents\Estudos\Angular + .Net\ProjetoFullStack_01> dotnet run
```



```
PROBLEMAS SAÍDA CONSOLE DE DEPURAÇÃO TERMINAL PORTAS RESULTADOS DA CONSULTA powershell - ClientApp + - [ ] [ ] ... | [ ] [X]
PS C:\Users\Maique1\Documents\Estudos\Angular + .Net\ProjetoFullStack_01\ClientApp> ng serve
```

→ Roda esses 2 comandos no terminal no vs code , 1 para rodar a API e outro para rodar servidor de teste do Angular.